Chương 2. Các mô hình dữ liệu

Ngôn ngữ SQL - Truy vấn cơ bản

Đỗ Thanh Hà

Khoa Trí tuệ nhân tạo
Học viện Công nghệ Bưu chính Viễn thông

Mục tiêu

- Viết được các truy vấn `SELECT` đơn giản trên một bảng.
- Hiểu **thứ tự xử lý** các mệnh đề
- Làm chủ **logic ba giá trị** và giá trị `NULL`.
- So sánh chuỗi, khớp mẫu, kiểu ngày–giờ.
- Sắp xếp kết quả và kiểm soát vị trí của `NULL`.
- Truy vấn nhiều bảng: tích Đề-các, các phép kết, các phép toán tập hợp.

Quy ước

Bài giảng dùng **SQL chuẩn** (ISO/IEC 9075) làm gốc. Chỗ nào một hệ quản trị khác chuẩn, slide ghi rõ bằng nhãn **[T-SQL]**, **[MySQL]**, **[Oracle]** hoặc **[PostgreSQL]**.

Nội dung

Phần 1 - Truy vấn một bảng

- Tổng quan về SQL
- Thứ tự xử lý các mệnh đề
- Phép chiếu (`SELECT`)
- Phép chọn (`WHERE`)
- So sánh chuỗi và khớp mẫu
- Kiểu ngày–giờ
- Giá trị `NULL` và logic ba giá trị
- Sắp xếp kết quả (`ORDER BY`)

Phần 2 - Truy vấn nhiều bảng

- Tích Đề-các
- Inner join, theta join
- Outer join (left / right / full)
- Phân biệt thuộc tính trùng tên
- Biến bộ (tuple variables)
- `UNION`, `INTERSECT`, `EXCEPT`
- Bài tập có lời giải

Tổng quan về SQL

- SQL (*Structured Query Language*) là ngôn ngữ phổ biến nhất để truy vấn và sửa đổi cơ sở dữ liệu quan hệ.
- Ra đời tại IBM đầu thập niên 1970 với tên *SEQUEL* (*Structured English Query Language*), dựa trên công trình về mô hình quan hệ của E. F. Codd. Tên bị rút gọn thành SQL vì lý do nhãn hiệu — nên vẫn thường được đọc là "sequel".
- Là ngôn ngữ **khai báo** (declarative): ta mô tả *cần gì*, không mô tả *làm thế nào*. Bộ tối ưu hóa của hệ quản trị tự chọn cách thực thi.
- Được chuẩn hóa bởi ANSI (1986) và ISO (1987), cập nhật nhiều lần từ đó đến nay.

Thực tế

Không hệ quản trị nào cài đặt đầy đủ 100% chuẩn, và mọi hệ đều có phần mở rộng riêng. "SQL" trên thực tế là một *họ* ngôn ngữ gần giống nhau

Các phiên bản chuẩn SQL

Năm	Tên	Nội dung chính
1986/87	SQL-86	Bản đầu tiên của ANSI, ISO phê chuẩn năm 1987.
1989	SQL-89	Sửa đổi nhỏ; bổ sung ràng buộc toàn vẹn tham chiếu.
1992	SQL-92	Sửa đổi lớn: `JOIN` tường minh, truy vấn con, `CASE`, kiểu ngày–giờ, `CAST`.
1999	SQL:1999	Truy vấn đệ quy, trigger, kiểu người dùng định nghĩa, biểu thức chính quy, tính năng hướng đối tượng.
2003	SQL:2003	**Hàm cửa sổ** (window functions), tính năng XML, cột tự sinh (`IDENTITY`).
2008	SQL:2008	Mở rộng hàm cửa sổ, `TRUNCATE`, `FETCH FIRST`.
2011	SQL:2011	Dữ liệu theo thời gian (temporal tables).
2016	SQL:2016	Hỗ trợ **JSON**, khớp mẫu hàng, hàm đa trị.
2023	SQL:2023	Kiểu dữ liệu `JSON`; truy vấn đồ thị thuộc tính (SQL/PGQ — khớp mẫu đồ thị trên bảng quan hệ sẵn có).

Mốc đáng nhớ nhất là **SQL-92**: phần lớn cú pháp dạy trong học phần này đến từ đó. Tuy vậy "thuộc chuẩn" không đồng nghĩa với "chạy ở mọi nơi" — `FULL OUTER JOIN`, `INTERSECT` và `EXCEPT` đều là SQL-92 mà MySQL mãi tới 8.0.31 mới có hai cái sau, còn cái đầu đến nay vẫn chưa.

Các ngôn ngữ con của SQL

Nhóm	Lệnh tiêu biểu	Chức năng
DQL	`SELECT`	Truy vấn dữ liệu
DML	`INSERT, UPDATE, DELETE`	Thao tác dữ liệu
DDL	`CREATE, ALTER, DROP, TRUNCATE`	Định nghĩa cấu trúc
TCL	`COMMIT, ROLLBACK, SAVEPOINT`	Kiểm soát giao dịch
DCL	`GRANT, REVOKE`	Kiểm soát quyền truy cập

- Nhiều tài liệu xếp `SELECT` vào DML thay vì tách riêng DQL - cả hai cách gọi đều được chấp nhận; điều quan trọng là `SELECT` *chỉ đọc*, không thay đổi dữ liệu.
- **Toàn bộ học phần này nằm trong** `SELECT` - lệnh phức tạp nhất và được dùng nhiều nhất của SQL.

Phần mở rộng riêng của từng hệ quản trị

- Mỗi hệ quản trị bổ sung một ngôn ngữ thủ tục riêng trên nền SQL: **T-SQL** (SQL Server, Sybase), **PL/SQL** (Oracle), **PL/pgSQL** (PostgreSQL), **SQL/PSM** (chuẩn ISO, MySQL bám theo).
- Chúng thêm biến cục bộ, cấu trúc điều khiển (`IF/ELSE`, `WHILE`), xử lý lỗi, thủ tục và hàm do người dùng viết.

Chú ý

Thường nghe nói "T-SQL là siêu tập của SQL, mọi câu lệnh SQL hợp lệ đều chạy được trong T-SQL". Điều này **không chính xác**: T-SQL mở rộng SQL ở nhiều chỗ nhưng cũng *thiếu* hoặc *viết khác* một số tính năng chuẩn.

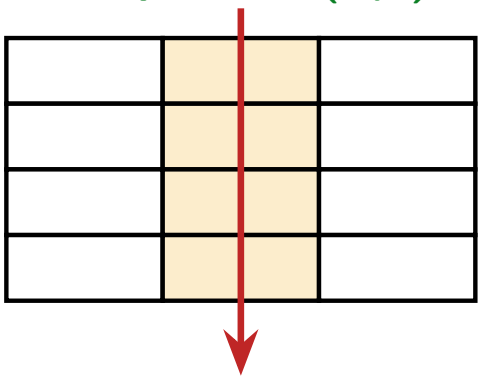
Chuẩn, Oracle 12c+, PostgreSQL	`FETCH FIRST 10 ROWS ONLY`
[T-SQL]	`TOP 10`, hoặc `ORDER BY ... OFFSET 0 ROWS FETCH FIRST 10 ROWS ONLY`
[MySQL] [PostgreSQL]	`LIMIT 10`

SQL Server *có* `FETCH FIRST` nhưng bắt buộc kèm cả `ORDER BY` lẫn `OFFSET` — không dùng đứng một mình được.

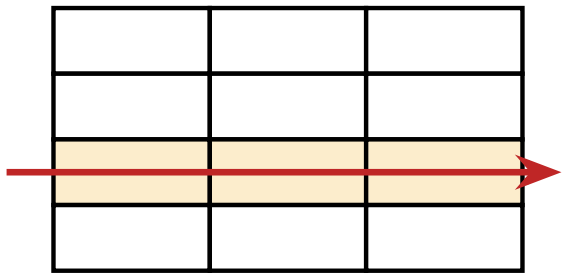
Quan hệ giữa các phương ngữ là *giao nhau*, không phải *bao hàm*.

Ba khả năng của câu lệnh SELECT

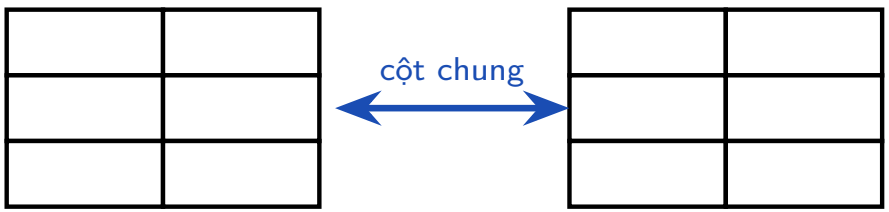
Phép chiếu (cột)



Phép chọn (hàng)



Phép kết (ghép hai bảng)



- **Chiếu** - chọn *cột* nào xuất hiện trong kết quả (danh sách sau `SELECT`).
- **Chọn** - chọn *hàng* nào được giữ lại (mệnh đề `WHERE`).
- **Kết** - ghép dữ liệu từ nhiều bảng qua giá trị chung.

Thuật ngữ

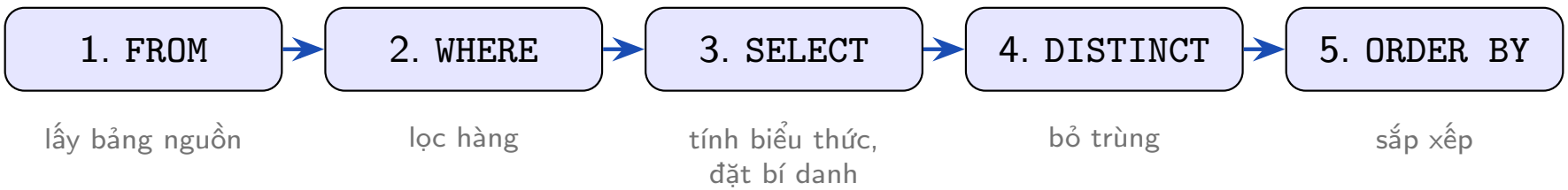
- **Từ khóa** (keyword): một thành phần SQL đơn lẻ. Ví dụ: `SELECT`, `FROM`, `WHERE`.
- **Mệnh đề** (clause): một phần của câu lệnh, gồm từ khóa và các đối số của nó. Ví dụ: `FROM employees` là một mệnh đề.
- **Câu lệnh** (statement): tổ hợp đầy đủ các mệnh đề tạo thành một đơn vị thực thi được. Ví dụ: `SELECT * FROM employees` là một câu lệnh.

Quy ước viết trong bài giảng

Từ khóa viết `IN HOA`, tên bảng và cột viết `thường`. SQL *không* phân biệt hoa thường ở từ khóa, nhưng quy ước này giúp đọc nhanh. Riêng **dữ liệu bên trong chuỗi** thì luôn phân biệt hoa thường (tùy quy tắc đối chiếu — collation — của cơ sở dữ liệu).

Thứ tự xử lý các mệnh đề

SQL *viết* theo một thứ tự nhưng *thực hiện* theo thứ tự khác.



Hai hệ quả

(1) `WHERE` chạy *trước* `SELECT`, nên **không dùng được bí danh cột trong** `WHERE`.
(2) `ORDER BY` chạy *sau* `SELECT`, nên **dùng được bí danh cột trong** `ORDER BY`.

Minh họa: vì sao bí danh chỗ được chỗ không

Không chạy được

```
SELECT price*23000 AS vnd
FROM   Products
WHERE  vnd > 1000000;
```

```
ERROR: column "vnd" does not exist
```

Khi `WHERE` chạy, mệnh đề `SELECT` chưa được tính nên bí danh `vnd` chưa tồn tại.

Cách sửa — lặp lại biểu thức

```
SELECT price*23000 AS vnd
FROM   Products
WHERE  price*23000 > 1000000;
```

Chạy được

```
SELECT price*23000 AS vnd
FROM   Products
ORDER BY vnd DESC;
```

`ORDER BY` chạy sau `SELECT` nên bí danh đã có.

Cũng chạy được — theo số thứ tự

```
SELECT name, price*23000 AS vnd
FROM   Products
ORDER BY 2 DESC;
```

Số 2 là vị trí cột trong danh sách `SELECT`. Ngắn nhưng dễ hỏng khi sửa danh sách cột — nên hạn chế dùng.

Cú pháp truy vấn SELECT cơ bản

```
SELECT [ ALL | DISTINCT ] { * | bieu_thuc [AS bi_danh], ... }
FROM   ten_bang
[WHERE dieu_kien]
[ORDER BY bieu_thuc [ASC | DESC], ...]
[OFFSET m ROWS] [FETCH FIRST n ROWS ONLY]
```

- `SELECT` — các cột/biểu thức xuất hiện trong kết quả.
  - `ALL` (mặc định): giữ các hàng trùng lặp.
  - `DISTINCT`: chỉ giữ các hàng duy nhất. Với `DISTINCT`, các giá trị `NULL` được coi là *bằng nhau* nên chỉ còn lại một.
- `FROM` — nguồn dữ liệu.
- `WHERE` — điều kiện lọc hàng (tương ứng phép chọn σ của đại số quan hệ).

Giới hạn số dòng trả về — cú pháp khác nhau theo hệ quản trị

Chuẩn SQL:2008, Oracle 12c+, PostgreSQL	`FETCH FIRST n ROWS ONLY`
[MySQL] [PostgreSQL]	`LIMIT n`
[T-SQL]	`SELECT TOP n ...`

Mẹo đọc và viết một truy vấn

- Đọc (và viết) theo đúng *thứ tự thực hiện*, không theo thứ tự chữ viết:
  1. `FROM` — dữ liệu lấy từ đâu?
  2. `WHERE` — giữ lại những hàng nào?
  3. `SELECT` — hiển thị cái gì, dưới dạng nào?
  4. `ORDER BY` — sắp xếp ra sao?
- Cách này đặc biệt hữu ích khi gỡ lỗi: nếu kết quả thiếu hàng, vấn đề gần như luôn nằm ở `FROM` hoặc `WHERE`; nếu thiếu/sai cột, vấn đề nằm ở `SELECT`.

Bảng dữ liệu dùng trong phần 1

Persons

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

Chọn tất cả các cột, và chọn một số cột

```
SELECT * FROM Persons;
```

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

```
SELECT LastName, FirstName
FROM Persons;
```

LastName	FirstName
Hansen	Ola
Svendson	Tove
Pettersen	Kari

SELECT * tiện khi thử nghiệm, nhưng tránh dùng trong chương trình

(1) Truyền về những cột không cần — tốn băng thông và bộ nhớ.
(2) Nếu ai đó thêm/xóa/đổi thứ tự cột trong bảng, chương trình đọc kết quả theo vị trí sẽ hỏng âm thầm.
(3) Che mất ý định: người đọc mã không biết truy vấn thực sự cần cột nào.

Đổi tên cột bằng bí danh

```
SELECT ProductCode, Name AS "Product's Name"
FROM   Products;
```

ProductCode	Product's Name
P01	Paper A4 EPSON
P02	Keyboard
P03	Mouse
P04	17" LCD Monitor
P05	21" LCD Monitor
P06	USB 8G
P07	USB 16G

- Bí danh chứa dấu cách hoặc ký tự đặc biệt phải được *bao bởi dấu định danh*.
- Chuẩn SQL dùng **dấu nháy kép** "...". Các hệ khác: [MySQL] dấu huyền `...`; [T-SQL] ngoặc vuông [...].
- **Phân biệt rõ:** nháy đơn '...' là *giá trị chuỗi*; nháy kép "..." là *tên đối tượng*. Nhầm hai thứ này là lỗi rất hay gặp.

Chiếu mở rộng - tạo cột tính toán

```
SELECT ProductCode, Name AS "Product's Name",
       Price AS "Price in USD", Price*23000 AS "Price in VND"
FROM   Products;
```

ProductCode	Product's Name	Price in USD	Price in VND
P01	Paper A4 EPSON	10	230,000
P02	Keyboard	15	345,000
P03	Mouse	15	345,000
P04	17" LCD Monitor	159	3,657,000
P05	21" LCD Monitor	219	5,037,000
P06	USB 8G	45	1,035,000
P07	USB 16G	60	1,380,000

Cột `"Price in VND"` **không** được thêm vào bảng `Products`; nó chỉ tồn tại trong kết quả của truy vấn này. Muốn lưu lại thì phải dùng `ALTER TABLE` hoặc cột sinh (`GENERATED` column) — nhưng lưu một giá trị tính được từ cột khác thường là thiết kế không tốt, vì tỷ giá thay đổi sẽ làm dữ liệu sai lệch.

Loại bỏ trùng lặp với DISTINCT

Persons — *bản mở rộng*

LastName	City
Hansen	Sandnes
Svendson	Sandnes
Pettersen	Stavanger
Nilsen	*NULL*
Olsen	*NULL*

Thêm hai hàng có `City` rỗng so với bảng ở các slide trước, để minh họa `NULL`.

```
SELECT City
FROM   Persons;
```

City
Sandnes
Sandnes
Stavanger
NULL
NULL

5 hàng

```
SELECT DISTINCT City
FROM   Persons;
```

City
Sandnes
Stavanger
NULL

3 hàng

DISTINCT và NULL

Trong `WHERE`, hai `NULL` so sánh với nhau cho *UNKNOWN* — *không* bằng nhau. Nhưng với `DISTINCT` (và `GROUP BY`, `UNION`), chuẩn SQL quy định các `NULL` được coi là **trùng nhau** nên chỉ còn lại một. Hai quy tắc khác nhau cho hai ngữ cảnh khác nhau.

Phép chọn - mệnh đề WHERE

- `WHERE` giữ lại các hàng làm điều kiện nhận giá trị **TRUE**. Hàng cho **FALSE** hoặc **UNKNOWN** đều bị loại.
- Tương ứng với toán tử chọn σ của đại số quan hệ.

Nhóm	Toán tử	Ghi chú
So sánh	`= <> < > <= >=`	`!=` là bí danh của `<>`; `<>` mới là chuẩn
Khoảng	`BETWEEN a AND b`	Bao gồm *cả hai* đầu mút
Tập hợp	`IN (v1, v2, ...)`	Cẩn thận khi danh sách có `NULL`
Mẫu	`LIKE`	Khớp mẫu chuỗi
Rỗng	`IS NULL, IS NOT NULL`	`= NULL` luôn sai
Logic	`AND OR NOT`	Logic *ba* giá trị

```
SELECT last_name, salary          FROM employees WHERE salary BETWEEN 2500
     AND 3500;
SELECT last_name, manager_id      FROM employees WHERE manager_id IS NULL;
SELECT last_name, job_id          FROM employees WHERE job_id NOT IN ('
    IT_PROG','SA_REP');
SELECT employee_id, last_name     FROM employees WHERE salary >= 10000
                                        AND job_id LIKE '%MAN%'
    ;
```

So sánh chuỗi

- Hai chuỗi bằng nhau nếu là cùng một dãy ký tự, theo quy tắc **đối chiếu** (collation) của cột — quy tắc này quyết định có phân biệt hoa thường và dấu hay không.
- Dùng được các toán tử `<` `>` `=` `<=` `>=` `<>`; thứ tự là thứ tự từ điển theo collation.

CHAR và VARCHAR

Có bỏ qua khoảng trắng đệm ở cuối hay không là do **collation**: `PAD SPACE` thì bỏ qua, `NO PAD` thì không — *không phải* do `CHAR` hay `VARCHAR`. Với MySQL dùng `utf8mb4_general_ci` (`PAD SPACE`), dòng cuối bảng dưới là TRUE. Kết quả trên PostgreSQL 16:

Kiểu cột, giá trị `'abc'`	Biểu thức	Kết quả
`CHAR(10)`	`c = 'abc'`	TRUE
`CHAR(10)`	`c = 'abc  '`	TRUE
`VARCHAR(10)`	`v = 'abc'`	TRUE
`VARCHAR(10)`	`v = 'abc  '`	FALSE

Chuỗi Unicode: chuẩn SQL dùng `N'...'`; chú ý `N` đứng *trước* dấu nháy:

```
SELECT * FROM Customers WHERE Address <> N'Binh Chanh';    -- dung
-- SAI cu phap:          WHERE Address <> 'N'Binh Chanh'
```

[T-SQL] bắt buộc `N'...'` khi so sánh với `NVARCHAR`. [PostgreSQL] [MySQL] dùng `UTF-8` mặc định nên `'...'` là đủ.

Khớp mẫu - LIKE

Dạng: `s LIKE p` (và `s NOT LIKE p`)

Ký tự	Khớp với	Ví dụ
%	**không hoặc nhiều** ký tự bất kỳ	'Star%' khớp *Star*, *Star Wars*
_	**đúng một** ký tự bất kỳ	'_____' khớp mọi chuỗi dài đúng 5 ký tự

```
-- Phim co tua bat dau bang 'Star' va mot dau cach
SELECT title FROM Movies WHERE title LIKE 'Star %';
-- Phim co tua dai dung 5 ky tu
SELECT title FROM Movies WHERE title LIKE '_____';
-- Tim dung dau phan tram: dung ESCAPE
SELECT title FROM Movies WHERE title LIKE '%100\%%' ESCAPE '\';
```

Lưu ý

(1) % khớp cả *chuỗi rỗng*, nên 'Star%' khớp luôn chính từ *Star*.
(2) LIKE có phân biệt hoa thường hay không là do **collation** quyết định, không do bản thân LIKE. [PostgreSQL] có thêm ILIKE để khớp không phân biệt hoa thường.
(3) Mẫu bắt đầu bằng % **không dùng được chỉ mục B-tree**, nên thường phải quét toàn bảng. (Chỉ mục trigram GIN vẫn phục vụ được.)

Khớp một trong nhiều ký tự

Bài toán: liệt kê các phim có tựa bắt đầu bằng `'S'` hoặc `'G'`.

Cách viết thường gặp — không khả chuyển

```
SELECT title FROM Movies
WHERE  title LIKE '[S,G]%';
```

Hai vấn đề:

(1) Lớp ký tự `[...]` **không có** trong chuẩn SQL — chỉ [T-SQL] và MS Access hỗ trợ.

(2) Ngay trong T-SQL, `[S,G]` là tập *ba* ký tự `S`, `,`, `G` — khớp cả tựa bắt đầu bằng dấu phẩy.

Viết đúng phải là `'[SG]%'`.

Cách viết chuẩn — chạy ở mọi hệ

```
SELECT title FROM Movies
WHERE  title LIKE 'S%'
   OR  title LIKE 'G%';
```

Hoặc dùng biểu thức chính quy của chuẩn SQL:1999:

```
WHERE title SIMILAR TO '(S|G)%'
```

Kiểu ngày và giờ

- Chuẩn SQL có ba kiểu: `DATE`, `TIME`, `TIMESTAMP` (và biến thể `WITH TIME ZONE`).
- So sánh được bằng các toán tử quen thuộc `< > <= >= = <>`, và trừ nhau cho ra `INTERVAL`.

Kiểu	Dạng viết chuẩn (ISO 8601)	Ghi chú
`DATE`	`DATE '1948-05-14'`	Luôn `YYYY-MM-DD`
`TIME`	`TIME '15:00:02'`	Có thể kèm phần lẻ giây
`TIMESTAMP`	`TIMESTAMP '1948-05-14 12:00:02'`	Ngày + giờ

Luôn dùng dạng `YYYY-MM-DD`

Chuỗi `'05/14/1948'` hay `'14 May 1948'` *phụ thuộc vào thiết lập vùng miền* của máy chủ. Cùng một câu lệnh có thể chạy đúng trên máy này và sai trên máy khác — hoặc tệ hơn, chạy "đúng" với *tháng và ngày bị hoán đổi*: `'03/04/2024'` là 3 tháng 4 hay 4 tháng 3?

Dạng `YYYY-MM-DD` của ISO 8601 không mơ hồ và được mọi hệ quản trị hiểu đúng.

```
SELECT * FROM Orders WHERE OrderDate >= DATE '2000-05-01'
                       AND OrderDate <  DATE '2000-06-01';
```

Giá trị NULL

- `NULL` là **ký hiệu** biểu thị sự vắng mặt của giá trị, *không phải* một giá trị. Nó không thuộc miền giá trị nào.
- `NULL` mang nhiều ý nghĩa khác nhau: *chưa biết*, *không áp dụng*, hoặc *bị che giấu*. SQL không phân biệt ba trường hợp này.

Phép toán	Kết quả	Vì sao
`NULL + 10`	`NULL`	Số học với ẩn số cho ẩn số
`NULL = NULL`	*UNKNOWN*	Hai cái chưa biết, không kết luận được
`NULL <> 10`	*UNKNOWN*	Tương tự
`'abc' || NULL`	`NULL`	Nối chuỗi cũng lan truyền `NULL`

```
SELECT NULL + 10;                    -- ket qua: NULL
SELECT 1 WHERE NULL <> 10;           -- 0 dong: dieu kien UNKNOWN, khong phai
    TRUE
SELECT last_name FROM employees WHERE manager_id IS NULL;   -- dung
-- SAI:                              WHERE manager_id = NULL      -- luon 0
    dong
```

Cách kiểm tra *trực tiếp* là `IS NULL` / `IS NOT NULL`. Chuẩn SQL còn có `IS [NOT] DISTINCT FROM` coi hai `NULL` là bằng nhau, và `COALESCE` để thay `NULL` bằng giá trị mặc định.

Logic ba giá trị

p	q	p OR q	p AND q	$p = q$
T	T	T	T	T
T	F	T	F	F
T	U	T	U	U
F	T	T	F	F
F	F	F	F	T
F	U	U	F	U
U	T	T	U	U
U	F	U	F	U
U	U	U	U	U

p	NOT p
T	F
F	T
U	U

- T = TRUE, F = FALSE, U = UNKNOWN.
- Hai ô tô xanh đáng nhớ: `TRUE OR U = TRUE` và `FALSE AND U = FALSE` — khi một vế đã quyết định kết quả thì vế chưa biết không còn quan trọng.
- Ô tô đỏ là nguồn gốc mọi rắc rối: `NOT UNKNOWN` vẫn là `UNKNOWN`, nên **phủ định không làm hàng bị loại trở thành hàng được chọn**.

Hệ quả 1: truy vấn "bao trùm" vẫn bỏ sót hàng

```
SELECT * FROM Movies WHERE length <= 120 OR length > 120;
```

Câu hỏi: truy vấn này có trả về *mọi* bộ phim không?

Movies

title	length
Star Wars	124
Gone With The Wind	231
Shrek	90
Ghost	127
Alien	117
Mystery Film	*NULL*

Không.

Với *Mystery Film*:

- `NULL <= 120` → UNKNOWN
- `NULL > 120` → UNKNOWN
- `U OR U` → UNKNOWN

`WHERE` chỉ giữ hàng cho TRUE, nên hàng này bị loại.

```
-- Muon lay het:
WHERE length <= 120
   OR length >  120
   OR length IS NULL;
```

Bài học: luật "loại trừ bên thứ ba" của logic hai giá trị ($p \vee \neg p$ luôn đúng) *không còn đúng* trong SQL.

Hệ quả 2: NOT IN gặp NULL

Bảng J

x
1
2
NULL

```
SELECT 1 WHERE 5 NOT IN (SELECT x FROM J);
```

Trực giác nói: 5 không có trong $\{1, 2, \mathit{NULL}\}$ nên phải trả về 1 dòng.

Thực tế: 0 dòng.

Vì sao. `5 NOT IN (1,2,NULL)` được khai triển thành:

$$\neg(5 = 1 \ \vee \ 5 = 2 \ \vee \ 5 = \texttt{NULL}) \ = \ \neg(\mathsf{F} \vee \mathsf{F} \vee \mathsf{U}) \ = \ \neg\mathsf{U} \ = \ \mathbf{U}$$

Điều kiện là UNKNOWN nên hàng bị loại.

Đặc điểm

Cả `5 IN (...)` lẫn `5 NOT IN (...)` đều trả về **0 dòng**. Một truy vấn và phủ định của nó cùng cho tập rỗng.

Cách phòng tránh: dùng `NOT EXISTS` thay cho `NOT IN` khi truy vấn con có thể sinh `NULL`, hoặc thêm `WHERE x IS NOT NULL` vào truy vấn con.

Sắp xếp kết quả - ORDER BY

```
SELECT ... FROM ... [WHERE ...]
ORDER BY bieu_thuc1 [ASC | DESC] [NULLS FIRST | NULLS LAST], bieu_thuc2
    ...
```

- `ORDER BY` đứng **sau** mọi mệnh đề lọc; chỉ còn `OFFSET/FETCH` (hay `LIMIT`) được phép đứng sau nó. `ASC` là mặc định.
- Cột sắp xếp chỉ định bằng: tên cột, **bí danh**, biểu thức, hoặc **số thứ tự** trong danh sách `SELECT`.
- Nhiều cột: sắp theo cột đầu trước; hàng bằng nhau ở cột đầu mới xét cột sau.

```
SELECT * FROM Movies ORDER BY length DESC;                    -- theo ten
    cot
SELECT Name, Price*23000 AS vnd
FROM   Products      ORDER BY vnd DESC;                       -- theo bi
    danh
SELECT * FROM Movies ORDER BY length DESC, title ASC;         -- nhieu cot
```

Không có ORDER BY thì *không có* thứ tự

Quan hệ là một *tập hợp* — không có thứ tự nội tại. Hệ quản trị tự do trả về theo thứ tự bất kỳ, và thứ tự đó có thể đổi khi dữ liệu lớn lên hoặc kế hoạch thực thi đổi. **Không bao giờ** dựa vào thứ tự "tự nhiên".

NULL nằm ở đâu khi sắp xếp?

Chuẩn SQL **không quy định** - mỗi hệ quản trị chọn một kiểu. Đây là nguồn lỗi khi chuyển ứng dụng giữa các hệ.

Hệ quản trị	`ORDER BY x ASC`	`ORDER BY x DESC`
PostgreSQL, Oracle	`NULL` ở **cuối**	`NULL` ở **đầu**
SQL Server, MySQL, SQLite	`NULL` ở **đầu**	`NULL` ở **cuối**

Cách viết không phụ thuộc hệ quản trị — chỉ định tường minh (chuẩn SQL:2003):

```
SELECT x FROM N ORDER BY x ASC NULLS LAST;
```

PostgreSQL và Oracle hỗ trợ trực tiếp. [MySQL] [T-SQL] chưa có, phải mô phỏng:

```
ORDER BY CASE WHEN x IS NULL THEN 1 ELSE 0 END, x;
```

Kiểm chứng trên PostgreSQL 16 với dữ liệu $\{3, \mathit{NULL}, 1\}$:

Truy vấn	Kết quả
`ORDER BY x ASC`	1, 3, *NULL*
`ORDER BY x DESC`	*NULL*, 3, 1

Truy vấn từ nhiều bảng

Mục tiêu phần 2

1. Truy vấn dữ liệu nằm trên nhiều quan hệ.
2. Phân biệt các thuộc tính trùng tên.
3. Hiểu vai trò của bí danh cột và **bí danh bảng** (biến bộ).
4. Phân biệt **inner join** và các loại **outer join**.
5. Sử dụng các phép toán tập hợp: `UNION`, `INTERSECT`, `EXCEPT`.

Hai bảng dùng xuyên suốt

`Customers(`<u>`CustomerID`</u>`, Name, Address, Phone, Email)`
`Orders(`<u>`OrderID`</u>`, OrderDate, `*`CustomerID`*`)`

Dữ liệu mẫu và tích Đề-các

Customers (6 hàng)

CustomerID	Name	Address	Phone	Email
C01	Nguyễn Thị Bé	Tân Bình	08457895	bnt@yahoo.com
C02	Lê Hoàng Nam	Bình Chánh	09878987	namle@abc.vn
C03	Trần Thị Chiêu	Tân Bình	08457895	*NULL*
C04	Mai Thị Quế Anh	Bình Chánh	*NULL*	*NULL*
C05	Lê Văn Sáng	Quận 10	*NULL*	sanglv@vnn.vn
C06	Trần Hoàng Khải	Tân Bình	08457897	*NULL*

Orders (7 hàng)

OrderID	OrderDate	CustomerID
Or001	2000-05-12	C01
Or002	2000-05-25	C02
Or003	2000-05-25	C01
Or004	2000-05-25	C04
Or005	2000-05-26	C04
Or006	2000-02-06	C03
Or007	2000-06-22	C04

C05 và C06 **chưa có đơn hàng nào** - điểm mấu chốt cho phần outer join sau này.

Tích Đề-các `Customers` $\times$ `Orders` có $6 \times 7 = 42$ hàng - mỗi khách ghép với *mọi* đơn hàng, kể cả đơn của người khác. Trong 42 hàng đó chỉ có **7** hàng "có nghĩa" (hàng mà `Customers.CustomerID = Orders.CustomerID`).

Tích Đề-các: khi nào vô tình tạo ra

- Tích Đề-các hình thành khi:
  - Quên điều kiện kết nối trong `WHERE`;
  - Điều kiện kết nối không hợp lệ (nối sai cột);
  - Kết nối n bảng mà có ít hơn $n - 1$ điều kiện nối.
- Kết quả: mọi hàng của bảng này ghép với mọi hàng của bảng kia.

Thiếu điều kiện nối — 42 hàng

```
SELECT * FROM Customers, Orders;
```

Có điều kiện nối — 7 hàng

```
SELECT * FROM Customers C, Orders O
WHERE  C.CustomerID = O.CustomerID;
```

Chú ý:

Tích Đề-các không báo lỗi — nó chỉ *chạy rất lâu* và trả về rất nhiều hàng. Hai bảng mỗi bảng 100.000 hàng cho 10^{10} hàng kết quả.

Phòng tránh: viết kết nối bằng cú pháp tường minh `JOIN ... ON`. Trên PostgreSQL, Oracle và SQL Server, quên mệnh đề `ON` là *lỗi cú pháp* nên bị bắt ngay. [MySQL] là ngoại lệ: `ON` là tùy chọn, `A JOIN B` vẫn chạy và cho tích Đề-các. Lưu ý `ON` nối *sai cột* thì không hệ nào bắt được.

Inner join - minh họa

CLASS

CLASS_ID	CLASS_NAME
106	Lop 106
107	Lop 107
201	Lop 201
202	Lop 202

Lớp 201, 202 **chưa có sinh viên**.

STUDENT

CLASS_ID	ID	NAME
106	1	A
106	2	B
107	3	C
107	4	D
NULL	5	E
NULL	6	F
NULL	7	G
NULL	8	H

Sinh viên E–H **chưa được xếp lớp**.

Hai nhóm hàng tô vàng là những hàng **không có đối tác** ở bảng kia. Chúng quyết định sự khác nhau giữa inner join và các loại outer join.

Inner join diễn đạt bằng đại số quan hệ

Định nghĩa

$$A \bowtie_\theta B \;=\; \sigma_\theta\,(A \times B)$$

Với kết nối bằng trên `class_id`:

$$\text{CLASS} \bowtie \text{STUDENT} \;=\; \sigma_{\text{CLASS.class_id} = \text{STUDENT.class_id}}(\text{CLASS} \times \text{STUDENT})$$

Lưu ý ký hiệu: toán tử chọn là σ (sigma). Ký hiệu δ là toán tử *loại bỏ trùng lặp*.

Hai cách viết tương đương trong SQL:

Cú pháp cũ (SQL-86)

```
SELECT *
FROM   Class A, Student B
WHERE  A.class_id = B.class_id;
```

Cú pháp tường minh (SQL-92) — **nên dùng**

```
SELECT *
FROM   Class A
       INNER JOIN Student B
       ON A.class_id = B.class_id;
```

Cú pháp `JOIN ... ON` tách bạch *điều kiện nối* (trong `ON`) khỏi *điều kiện lọc* (trong `WHERE`), và là cú pháp **bắt buộc** khi dùng outer join. Kết quả ở đây: **4 hàng**.

Theta join - điều kiện nối không phải dấu bằng

Bài toán: xếp loại sinh viên theo thang điểm.

Student

ID	MARKS	ID	MARKS
1	5	7	**10**
2	5	8	6
3	6	9	7
4	6	10	3
5	8	11	2
6	9		

11 sinh viên

Ranking — bản có lỗi

GRADE	MIN	MAX
A	8	10
B	5	8
C	0	5

Các khoảng **chồng lấn** ở đầu mút.

```
SELECT A.id, B.grade
FROM   Student A, Ranking B
WHERE  A.marks >= B.min
  AND  A.marks <  B.max;
```

$$\sigma_{A.marks \geq B.min \,\wedge\, A.marks < B.max}(A \times B)$$

Hai lỗi của thiết kế trên

(1) Mất dữ liệu ở biên. Với `< MAX`, loại A là khoảng $[8, 10)$, nên sinh viên số 7 đạt **10 điểm không được xếp loại nào**. 11 sinh viên vào, chỉ 10 ra.

(2) Đổi thành `<= MAX` lại hỏng kiểu khác. Khi đó điểm 5 thuộc cả B lẫn C, điểm 8 thuộc cả A lẫn B — 11 sinh viên cho **14 hàng** kết quả.

Theta join - thiết kế đúng

Nguyên nhân gốc: bảng `Ranking` định nghĩa các khoảng *chồng lấn* (`MAX` của loại này bằng `MIN` của loại kia). Sửa dữ liệu, không sửa truy vấn.

Ranking — bản đúng

GRADE	MIN	MAX
A	8	10
B	5	7
C	0	4

Các khoảng **rời nhau** và phủ kín mọi *điểm nguyên* 0..10.

```
SELECT A.id, A.marks, B.grade
FROM   Student A
       JOIN Ranking B
       ON A.marks BETWEEN B.min AND B.max;
```

`BETWEEN` bao gồm cả hai đầu mút, nên không còn điểm nào lọt ra ngoài.

Kiểm chứng: 11 sinh viên vào $\rightarrow$ **11** hàng ra, mỗi người đúng một loại.

ID	MARKS	GRADE	ID	MARKS	GRADE
1	5	B	7	10	A
2	5	B	8	6	B
3	6	B	9	7	B
4	6	B	10	3	C
5	8	A	11	2	C
6	9	A			

Bài học chung: khi nối theo khoảng, luôn hỏi *có giá trị nào không rơi vào khoảng nào không?* và *có giá trị nào rơi vào hai khoảng không?*

Lưu ý: cách sửa trên chỉ đúng vì `MARKS` là **số nguyên**. Nếu điểm có phần thập phân thì 4,5 lại không thuộc loại nào — khi đó phải dùng khoảng *nửa mở* $[\,mn,\ mx\,)$ ghép khít nhau.

Outer join - vấn đề cần giải quyết

- Hàng không thỏa điều kiện nối sẽ **không xuất hiện** trong kết quả inner join. Những hàng này gọi là **hàng treo** (dangling tuples).
- Trong dữ liệu: lớp 201, 202 (không có sinh viên) và sinh viên E–H (không có lớp) đều là hàng treo.
- `OUTER JOIN` giữ lại các hàng treo và điền `NULL` vào phần thiếu.

Loại kết nối	Giữ lại hàng treo của	Số hàng kết quả
`INNER JOIN`	không giữ bên nào	4
`LEFT OUTER JOIN`	bảng bên trái (CLASS)	6
`RIGHT OUTER JOIN`	bảng bên phải (STUDENT)	8
`FULL OUTER JOIN`	cả hai bảng	10

Left outer join

```
SELECT A.class_id, A.class_name, B.class_id, B.id, B.name
FROM   Class A LEFT OUTER JOIN Student B ON A.class_id = B.class_id;
```

CLASS		STUDENT		
CLASS_ID	CLASS_NAME	CLASS_ID	ID	NAME
106	Lop 106	106	1	A
106	Lop 106	106	2	B
107	Lop 107	107	3	C
107	Lop 107	107	4	D
201	Lop 201	*NULL*	*NULL*	*NULL*
202	Lop 202	*NULL*	*NULL*	*NULL*

- **Mọi lớp đều xuất hiện**, kể cả lớp chưa có sinh viên nào.
- Sinh viên E–H **không** xuất hiện — họ là hàng treo của bảng *bên phải*, mà left join không giữ.
- Phần dữ liệu thiếu được điền `NULL`. Đây là `NULL` do *phép nối sinh ra*, không phải `NULL` có sẵn trong dữ liệu.

Right outer join

```
SELECT A.class_id, A.class_name, B.class_id, B.id, B.name
FROM   Class A RIGHT OUTER JOIN Student B ON A.class_id = B.class_id;
```

CLASS		STUDENT		
CLASS_ID	CLASS_NAME	CLASS_ID	ID	NAME
106	Lop 106	106	1	A
106	Lop 106	106	2	B
107	Lop 107	107	3	C
107	Lop 107	107	4	D
NULL	*NULL*	*NULL*	5	E
NULL	*NULL*	*NULL*	6	F
NULL	*NULL*	*NULL*	7	G
NULL	*NULL*	*NULL*	8	H

Mọi sinh viên đều xuất hiện, kể cả người chưa được xếp lớp. Lớp 201, 202 biến mất.

Ghi nhớ: `A LEFT JOIN B` $\equiv$ `B RIGHT JOIN A`. Trong thực tế người ta hầu như chỉ dùng `LEFT JOIN` và đổi thứ tự bảng khi cần — đọc dễ hơn vì bảng "chính" luôn đứng trước.

Full outer join

```
SELECT A.class_id, A.class_name, B.class_id, B.id, B.name
FROM   Class A FULL OUTER JOIN Student B ON A.class_id = B.class_id;
```

CLASS		STUDENT		
CLASS_ID	CLASS_NAME	CLASS_ID	ID	NAME
106	Lop 106	106	1	A
106	Lop 106	106	2	B
107	Lop 107	107	3	C
107	Lop 107	107	4	D
NULL	*NULL*	*NULL*	5	E
NULL	*NULL*	*NULL*	6	F
NULL	*NULL*	*NULL*	7	G
NULL	*NULL*	*NULL*	8	H
201	Lop 201	*NULL*	*NULL*	*NULL*
202	Lop 202	*NULL*	*NULL*	*NULL*

Vàng = phần thêm của `LEFT JOIN`; xanh = phần thêm của `RIGHT JOIN`. Tổng: 4+2+4 = 10 hàng.

[MySQL] **không hỗ trợ** `FULL OUTER JOIN`; phải mô phỏng bằng `LEFT JOIN UNION RIGHT JOIN`.

Phân biệt các thuộc tính trùng tên

- Khi truy vấn nhiều quan hệ, hai quan hệ có thể có thuộc tính trùng tên.
- SQL giải quyết bằng cách đặt **tên quan hệ và dấu chấm** trước tên thuộc tính: `R.A` là thuộc tính `A` của quan hệ `R`.

Ví dụ. Hai quan hệ cùng có `name` và `address`:

- `MovieStar(name, address, gender, birthdate)`
- `MovieExec(name, address, cert#, netWorth)`

Tìm các diễn viên và nhà sản xuất sống cùng địa chỉ:

```
SELECT MovieStar.name, MovieExec.name
FROM   MovieStar, MovieExec
WHERE  MovieStar.address = MovieExec.address;
```

Hai lưu ý

(1) Kết quả có *hai* cột cùng tên `name`. Nhiều công cụ chấp nhận nhưng khó dùng — nên đặt bí danh: `MovieStar.name AS star_name`.
(2) Chỉ *bắt buộc* dùng tiền tố khi tên bị nhập nhằng, nhưng **nên dùng cho mọi cột** trong truy vấn nhiều bảng: khi ai đó thêm cột mới vào một bảng, truy vấn không bị hỏng vì nhập nhằng bất ngờ.

Biến bộ (tuple variables)

- **Vấn đề:** khi so sánh các bộ trong *cùng một* quan hệ, tên quan hệ làm tiền tố không còn phân biệt được.
- **Giải pháp:** khai báo **biến bộ** cho mỗi lần xuất hiện của quan hệ trong `FROM`: `FROM Ten_Bang [AS] Ten_Bien`. `AS` là tùy chọn; [Oracle] *không* chấp nhận `AS` cho bí danh bảng.

Bài toán: cho `Movie(title, length)`, tìm các cặp phim có cùng độ dài.

Movie

title	length
Alien	117
Ghost	117
Shrek	90
Babe	90
Solo	90

```
SELECT M1.title, M2.title
FROM   Movie M1, Movie M2
WHERE  M1.length = M2.length
  AND  M1.title  < M2.title;
```

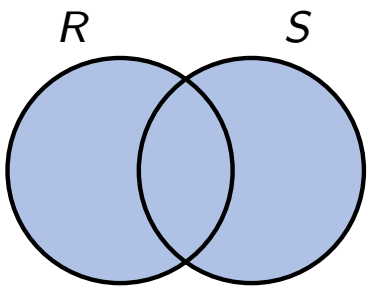
Kết quả **4 cặp**: (Alien, Ghost), (Babe, Shrek), (Babe, Solo), (Shrek, Solo).

Vai trò của điều kiện `M1.title < M2.title`

(1) Loại bỏ việc ghép một phim với *chính nó*.
(2) Loại bỏ cặp *hoán vị*: đã có (A, B) thì không hiện (B, A).

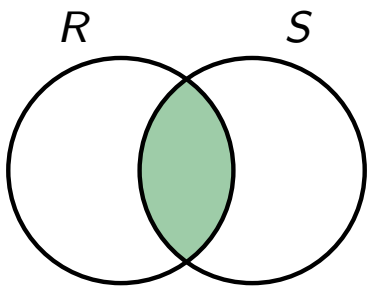
Thay < bằng <> thì được **8** hàng — đúng gấp đôi, vì mỗi cặp xuất hiện thêm một lần theo chiều ngược lại.

Các phép toán tập hợp



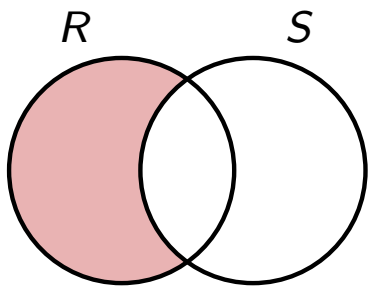
UNION

$R \cup S$



INTERSECT

$R \cap S$



EXCEPT

$R - S$

- Phần **tô màu** là kết quả của phép toán.
- Cả ba phép đều đòi hỏi hai vế **tương thích hợp nhất**: cùng số cột, các cột tương ứng có kiểu tương thích.
- Cả ba đều **loại bỏ trùng lặp** theo mặc định. Thêm `ALL` để giữ lại (`UNION ALL`, `INTERSECT ALL`, `EXCEPT ALL`).

UNION và UNION ALL

R
ID
1
2

S
ID
2
3

```
SELECT ID FROM R UNION     SELECT ID FROM S;
    -- 1, 2, 3
SELECT ID FROM R UNION ALL SELECT ID FROM S;
    -- 1, 2, 2, 3
```

- `UNION` loại bỏ trùng lặp — hệ quản trị phải sắp xếp hoặc băm toàn bộ kết quả để làm việc đó.
- `UNION ALL` giữ nguyên mọi hàng, nên **nhanh hơn**.
- **Quy tắc thực hành:** dùng `UNION ALL` trừ khi thật sự cần loại trùng. Nhiều người viết `UNION` theo thói quen và trả giá bằng hiệu năng.

Tên cột của kết quả lấy từ truy vấn *đầu tiên*

```
SELECT 1 AS alpha UNION ALL SELECT 2 AS beta;   -- cot ket qua ten la "
    alpha"
```

INTERSECT và EXCEPT

INTERSECT — phép giao

$$R \cap S = \{\, t \mid t \in R \ \wedge \ t \in S \,\}$$

VIP

ID
101
102
105

Oct_Orders

ID
101
103
105

```
SELECT ID FROM VIP
INTERSECT
SELECT ID FROM Oct_Orders;
```

Kết quả: **101, 105**

EXCEPT — phép hiệu

$$R - S = \{\, t \mid t \in R \ \wedge \ t \notin S \,\}$$

A (2023)

Ten_Phim
Batman
Superman
Joker

B (2024)

Ten_Phim
Superman
Joker
Iron Man

```
SELECT Ten_Phim FROM A
EXCEPT
SELECT Ten_Phim FROM B;
```

Kết quả: **Batman**

Ghi nhớ

`EXCEPT` **không giao hoán**: $R - S \neq S - R$. Ở ví dụ trên $B - A = \{Iron\ Man\}$.
Tên gọi theo hệ: `EXCEPT` — chuẩn SQL, PostgreSQL, SQL Server; `MINUS` — [Oracle]. [MySQL] có `EXCEPT` và `INTERSECT` từ 8.0.31, nhưng **không** có `MINUS`.

Bài tập 1 — biến bộ

Cho bảng `NhanVien(MaNV, TenNV, Luong, TenPhong)`.

Yêu cầu: viết truy vấn tìm các *cặp* nhân viên làm cùng phòng và có lương bằng nhau.

Lời giải:

```
SELECT NV1.TenNV, NV2.TenNV
FROM   NhanVien NV1, NhanVien NV2
WHERE  NV1.TenPhong = NV2.TenPhong
  AND  NV1.Luong    = NV2.Luong
  AND  NV1.MaNV     < NV2.MaNV;
```

Vì sao so sánh trên `MaNV` chứ không phải `TenNV`

Điều kiện cuối phải đặt trên **khóa chính**, không phải trên tên.

Nếu viết `NV1.TenNV < NV2.TenNV`, hai nhân viên *trùng tên* cùng phòng và cùng lương sẽ bị loại khỏi kết quả — vì với họ `TenNV < TenNV` là FALSE, đúng như khi so một người với chính mình. Truy vấn im lặng bỏ sót dữ liệu hợp lệ.

Dùng `MaNV` tránh được điều đó vì khóa chính bảo đảm duy nhất.

Bài tập 2 - phép toán tập hợp

Cho `SinhVien_Toan(MSSV)` và `SinhVien_Tin(MSSV)`.

1. Giỏi ít nhất một trong hai môn

```
SELECT MSSV FROM SinhVien_Toan
UNION
SELECT MSSV FROM SinhVien_Tin;
```

2. Giỏi cả hai môn

```
SELECT MSSV FROM SinhVien_Toan
INTERSECT
SELECT MSSV FROM SinhVien_Tin;
```

3. Giỏi Toán nhưng không giỏi Tin

```
SELECT MSSV FROM SinhVien_Toan
EXCEPT
SELECT MSSV FROM SinhVien_Tin;
```

4. Gộp, chấp nhận trùng

```
SELECT MSSV FROM SinhVien_Toan
UNION ALL
SELECT MSSV FROM SinhVien_Tin;
```

Nhận xét

Câu 4 là cách duy nhất trong bốn câu giữ được thông tin "sinh viên này giỏi *cả hai* môn" dưới dạng *hai* dòng.

Nếu hệ quản trị không có `INTERSECT` / `EXCEPT`, thay bằng `IN` / `NOT IN` hoặc `EXISTS` / `NOT EXISTS` - lưu ý cái bẫy `NOT IN` với `NULL` ở phần trước.

Bài tập 3 - OR và UNION có như nhau không?

Cho KhachHang(ID, Ten, ThanhPho).

Truy vấn A

```
SELECT ThanhPho FROM KhachHang
WHERE  Ten = 'An' OR Ten = 'Binh';
```

Truy vấn B

```
SELECT ThanhPho FROM KhachHang
WHERE  Ten = 'An'
UNION
SELECT ThanhPho FROM KhachHang
WHERE  Ten = 'Binh';
```

Trả lời: không. UNION loại trùng lặp, OR thì không.

KhachHang			A trả về	B trả về
ID	Ten	ThanhPho		
1	An	Ha Noi	Ha Noi	Ha Noi
2	Binh	Ha Noi	Ha Noi	Hue
3	An	Hue	Hue	
		số hàng	**3**	**2**

Khi nào hai truy vấn cho kết quả giống nhau

Chỉ khi kết quả của A vốn đã không có hàng trùng — ví dụ khi SELECT lấy cả khóa chính, hoặc khi thêm DISTINCT vào A.

Bài học: UNION mang theo một phép DISTINCT ẩn. Khi thay OR bằng UNION để tối ưu, phải dùng UNION ALL nếu muốn giữ nguyên ngữ nghĩa.

Tổng kết

- **Thứ tự xử lý** `FROM` → `WHERE` → `SELECT` → `DISTINCT` → `ORDER BY` giải thích gần như mọi hành vi khó hiểu của SQL — trong đó có việc bí danh dùng được ở `ORDER BY` nhưng không dùng được ở `WHERE`.
- `NULL` **không phải giá trị.** `WHERE` chỉ giữ hàng cho TRUE, nên hàng cho UNKNOWN bị loại — kể cả khi điều kiện "trông như" bao trùm mọi trường hợp. Luôn kiểm tra bằng `IS NULL`, và cảnh giác với `NOT IN`.
- **Inner join bỏ hàng treo; outer join giữ lại.** Chọn loại join theo câu hỏi nghiệp vụ: "mọi lớp, kể cả lớp trống" là left join, không phải inner join.
- **Viết theo chuẩn, ghi chú chỗ không chuẩn.** `LIKE '[SG]%'`, `TOP n`, `MINUS` đều là cú pháp riêng của một hệ.
- **Khi nối theo khoảng,** luôn kiểm tra hai đầu mút: có giá trị nào lọt ra ngoài mọi khoảng, hay rơi vào hai khoảng cùng lúc?

Phần tiếp theo

Truy vấn con, hàm tổng hợp (`COUNT`, `SUM`, `AVG`), `GROUP BY` và `HAVING`.

Tài liệu tham khảo

- H. Garcia-Molina, J. D. Ullman, J. Widom. *Database Systems: The Complete Book*, 2nd ed., Pearson, 2008 — Chương 6.
- R. Elmasri, S. B. Navathe. *Fundamentals of Database Systems*, 7th ed., Pearson, 2016 — Chương 6, 7.
- A. Silberschatz, H. F. Korth, S. Sudarshan. *Database System Concepts*, 7th ed., McGraw-Hill, 2020 — Chương 3, 4.
- C. J. Date. *SQL and Relational Theory: How to Write Accurate SQL Code*, 3rd ed., O'Reilly, 2015 — phân tích sâu về `NULL` và logic ba giá trị.
- ISO/IEC 9075:2023 — *Information technology — Database languages — SQL*.
- Tài liệu chính thức của PostgreSQL, MySQL, Oracle và SQL Server — tra cứu khi cần biết một tính năng có khả chuyển hay không.